

Day 1 → JVM, JRE, JDK And Compile → Build → Run

what is programming?

→ Imagine Ordering food through a robot waiter with zero imagination. you can't say "bring me something nice" - it won't get it.

you must say exact steps: "walk to table 4. Pick up plate A. Carry to kitchen".

programming = writing exact step-by-step instructions to a machine that does exactly what we say, nothing more.

what is JAVA?

- A language for writing a code / those instructions. Like English is a language used to write a letters.

Java's slogan: "write once, run anywhere"

- means code written once can run on windows, mac, or linux without rewriting

JDK vs JRE vs JVM

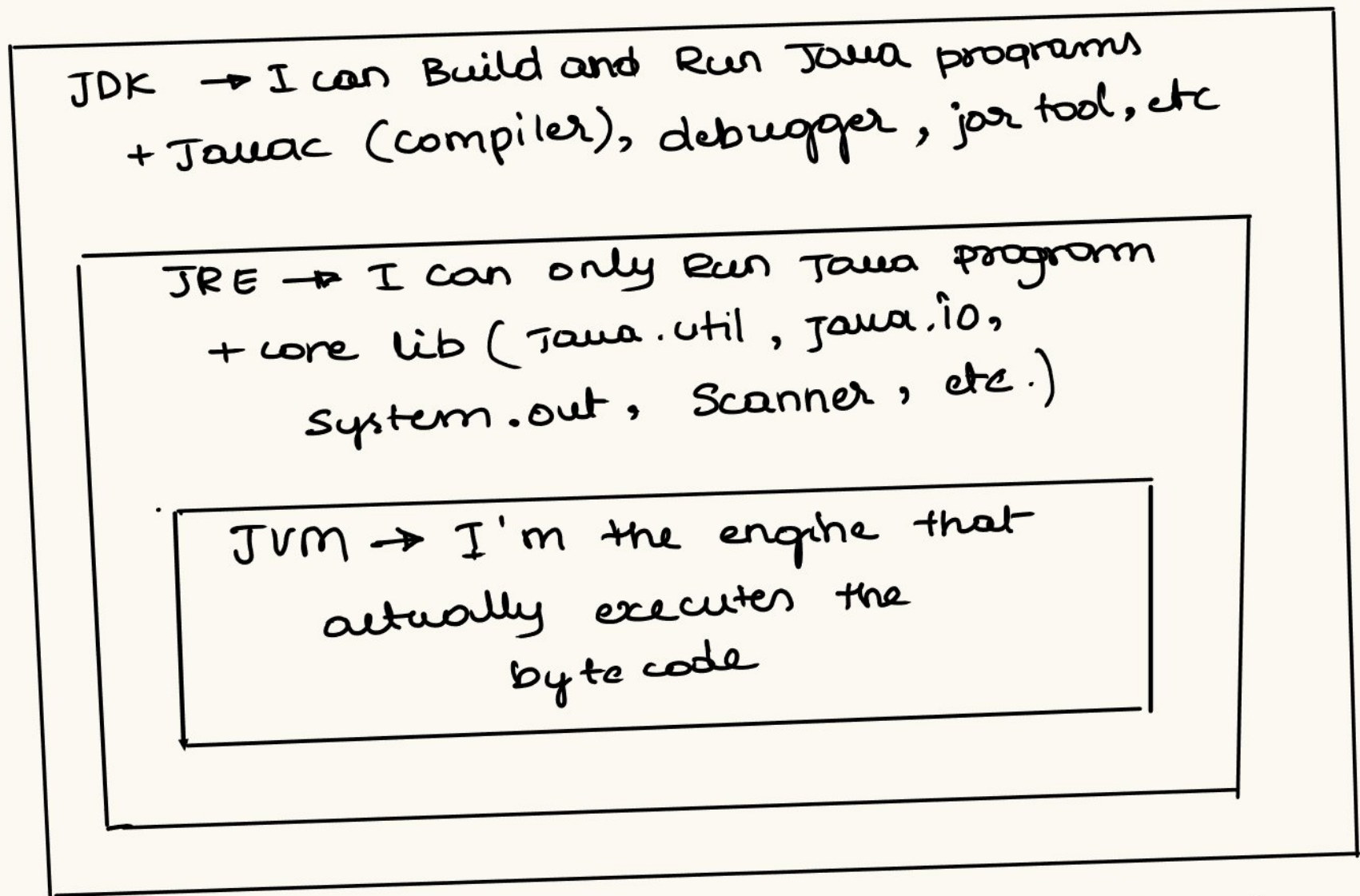


Kitchen Story

Term	Real Meaning	Kitchen Analogy
JDK	Everything to write & build code	The whole kitchen - stove, knives ingredients, recipe book
JRE	Everything to run already - built code	Just the dining table + stove, ready to serve.
JVM	The engine that execute our code	The chef at the stove, cooking recipe into real food.

Memory Trick :

JDK (Kitchen, Build) → JRE (dining table, serve) → JVM (chef, does the actual work)



→ To run a program, we need an engine (JVM) plus standard libraries every program calls (like System.out.println). This combo = JRE

→ To build a program, you need everything JRE has + extra tools only developers need - the compiler (Javac), debugger, javadoc, jar packager. This comb = JDK

Q. Interviewer: "why can't you compile Java code with just JRE installed?"

→ Because JRE deliberately only includes the JVM + runtime libraries needed to execute already-compiled bytecode - it leaves out Javac, since a regular user running an app doesn't need to build one. JDK is JRE + compiler and other dev tools.

→ JVM is not a tool it's a execution Engine (a part of Java)
bundled inside the JDK

→ IntelliJ / Eclipse are an IDE (Integrated Development Environment) - a separate third party app. It's not a part of Java at all.

Manager = IntelliJ / Eclipse CEO = us (the developer)

Employees = JVM

→ you (CEO) says "run this".

→ Manager (IntelliJ) doesn't do the running instead he tells to Employees.

"Hey employee, here's the bytecode, go execute it."

→ Employee (JVM / java.exe) does 100% work
- loading, verifying, running and managing memory (garbage collections) everything.

→ Employee hands the result to manager.

→ Manager (IntelliJ) presents it nicely in the Run window - as if he did the work, but really it's just relaying on the employee's output.

idea64.exe → IntelliJ

java.exe → JVM

Q. Interviewer:

"Are IntelliJ and JVM is the same thing?"

→ No - IntelliJ is an IDE, a development tool I use to write and click-to-run code, under the hood it just invokes javac then java - that java process is a separate, independent program which is the JVM. I could run the exact same compiled class from a plain command line with no IDE at all."

Compile → Build → Run

- **Compile**: a translator turns a English language (Main.java) into universal cooking-code (Main.class byte code) every kitchen understands.

One file in, one file out.

- **Build**: the whole prep - compile everything, gather ingredients (libraries), box it up ready to serve. Right now, for your tiny project, build = compile.
later (Spring Boot), build also means packaging into one box (.jar) with a label saying "start here"

- **Run**: the chef (JVM) actually performs the recipe live, right now, producing real output for a real customer.

Visual - what happens when you click run in IntelliJ

Main.java (your English recipe)



javac (JDK's compiler)
this is the compile step

Main.class (bytecode - universal recipe card)



JVM loads + verifies it
this is part of BUILD/RUN handoff

JVM finds: `Public static void main(String[] args)`



JVM executes it, line by line
this is the Run step

Output shown in Run window

Tips and Tricks :

- Java is both compiled and interpreted -
compiled once to bytecode, then the JVM's JIT
(Just-In-Time) compiler turns that into native
machine code while running.

- Get the main method signature wrong
(wrong case, missing static)

→ "Main method not found", which is a
launch error, not a compiler error. - compiling still
succeeded fine.

→ Grammar mistakes like missing Semi colon

program never starts → compile time error

→ Grammar and all fine but still something

breaks mid execution → runtime error / Exceptions

Q. Interviewer: "Is Java compiled or interpreted"

→ It actually both. When I compile my code with
Javac, it produces platform-independent bytecode -

not native machine code yet. Then at runtime, the
JVM's JIT compiler translates that bytecode into
native instructions for the specific machine it's running
on. That's what gives Java its "write once, run
anywhere" behavior, while still staying fast.

Q. Interviewer : " what is the difference between JDK, JRE and JVM ? "

→ JDK is the full development kit - it has the compiler and tools to build Java apps. JRE is just what's needed to run a built app - it includes the JVM plus core libraries.

JVM is the actual engine inside JRE that loads and executes the bytecode.

So

JDK > JRE > JVM